

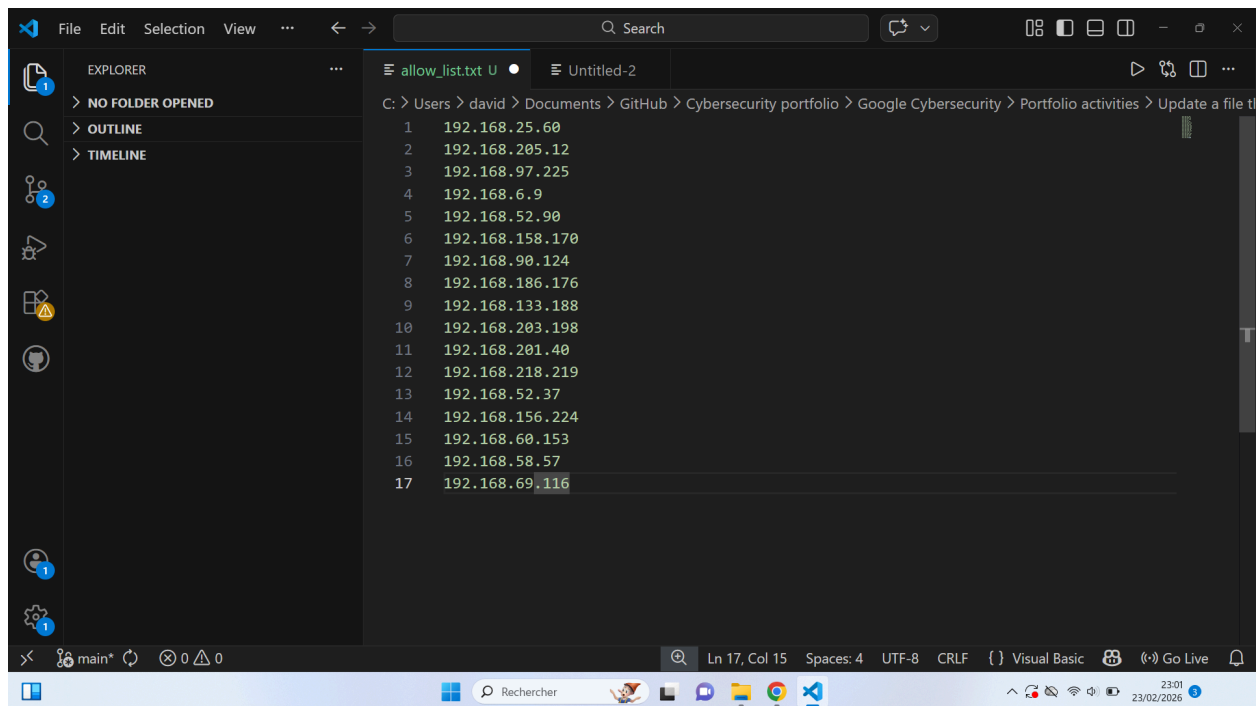
Algorithm for IPs detection and removal in Python

Project description

In this project, I developed an algorithm that parses a file containing IP addresses that are allowed to access restricted content and removes addresses that no longer have access => access management.

Afterwards, I created another function that I used to check if unauthorized ids were present and how many were present in the allowed_id list and if TRUE , I executed the function and then checked afterwards if those id were removed.

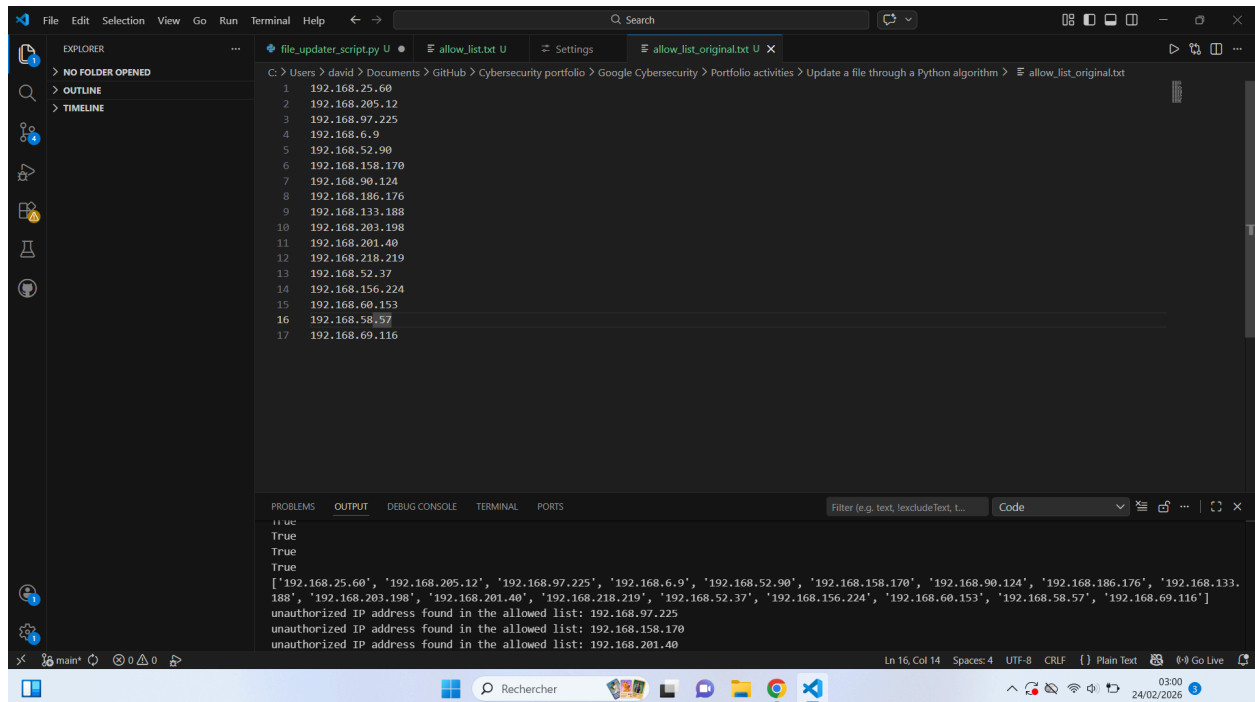
I have created a file named allow_list.txt containing the allowed IDs in the project's folder.



The screenshot shows a code editor with a dark theme. The Explorer panel on the left shows a file named 'allow_list.txt'. The main editor area displays the contents of this file, which is a list of 17 IP addresses, each on a new line. The IP addresses are: 192.168.25.60, 192.168.205.12, 192.168.97.225, 192.168.6.9, 192.168.52.90, 192.168.158.170, 192.168.90.124, 192.168.186.176, 192.168.133.188, 192.168.203.198, 192.168.201.40, 192.168.218.219, 192.168.52.37, 192.168.156.224, 192.168.60.153, 192.168.58.57, and 192.168.69.116. The status bar at the bottom indicates the current position is Line 17, Column 15, with 4 spaces, UTF-8 encoding, and CRLF line endings.

```
1 192.168.25.60
2 192.168.205.12
3 192.168.97.225
4 192.168.6.9
5 192.168.52.90
6 192.168.158.170
7 192.168.90.124
8 192.168.186.176
9 192.168.133.188
10 192.168.203.198
11 192.168.201.40
12 192.168.218.219
13 192.168.52.37
14 192.168.156.224
15 192.168.60.153
16 192.168.58.57
17 192.168.69.116
```

And another one named allow_list_original.txt to keep the data untouched for testing the code multiple times



Creating a function

I created the function with two parameters, one is the original IPs list of allowed IPs and the other is the list of IDs to be removed.

```
def remove_ip_from_file(allowed_ip_file, array):
```

Open the file that contains the allow list

To open the file containing the allow list, I used the with open command that takes a filename and r,w or a as parameters to know if I want to read, write a new file or write additional informations.

```
# Open the file in read mode and read its content.

with open(allowed_ip_file, 'r') as file:
```

Read the file contents

I used the file.read() command to store the file content in a variable

```
# Store the content of the file in a variable called `file_content`.

file_content = file.read()
```

Convert the string into a list

And then used the split() built in function for list to convert the string into a list

```
# Split the content of the file into a list of IDs and store it in a
variable called `list_of_ids`.

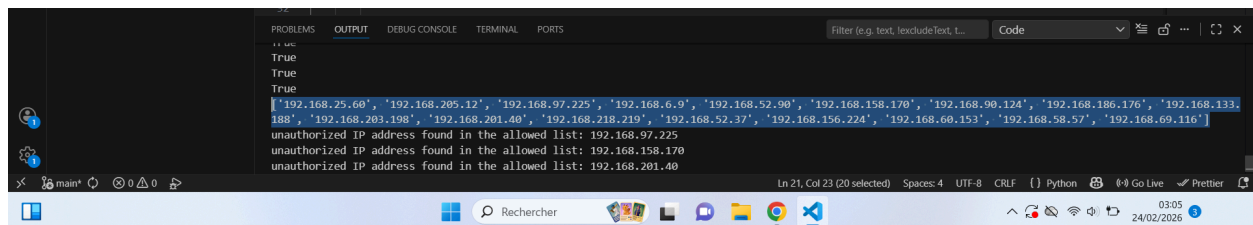
list_of_ips = file_content.split()
```

Verification

I used a print() function to verify if my code was working well

```
print(list_of_ips)
```

I had the confirmation in the console log.



Iterate through the remove list

I used the for in loop to iterate through each item in the id to be removed list and apply a condition

```
# Loop through each IP address in the array.

for ip in array:
```

Remove IP addresses that are on the remove list

After checking if each item is also present in the allowed IPs list , i remove them from the allowed list

```
if ip in list_of_ips:

    # If it is, remove it from the list.
    print("unauthorized IP address found in the allowed list: " +
ip)

    list_of_ips.remove(ip)
else:
    print("IP address not found in the allowed list: " + ip)
```

I added print statement for more clarity during developpement

Convert the list back into a string using join()

I ran into a problem initially with all the ids being next to each other
So i added \n

```
list_of_ips = "".join(list_of_ips)

list_of_ips = "\n".join(list_of_ips)
```

Update the file with the revised list of IP addresses

```
# Open the file in write mode and write the updated list of IDs back to
the file.

with open(allowed_ip_file, 'w') as file:
    file.write(list_of_ips)
```

I reopened the file and rewrote the file using the new list of values updated without the removed ones.

Verification function

Next, I wanted to add a function that allows me to check if the initially detected values were actually removed after detection to confirm that my function was working correctly:

I took the same arguments, it's the same function only that instead of removing ids, I print a boolean in the console and also return the boolean.

```
def is_ip_in_file(allowed_ip_file, array):
    with open(allowed_ip_file, 'r') as file:
        file_content = file.read()
    list_of_ips = file_content.split()
    for ip in array:
        if ip in list_of_ips:
            print("True")
            return
        else:
            print("False")
            return
```

I checked before to see if there was unauthorized ids, used the removing function and then rechecked if it removed them.

```
# check if not allowed ids exist in the allowed list

is_ip_in_file("allow_list.txt", remove_list)

# Call the `remove_id_from_file` function, passing in the file name and
the `remove_list` as arguments if the IP addresses in the `remove_list`
are still present in the allowed list file.

remove_ip_from_file("allow_list.txt", remove_list)
```

```
# check if not allowed ids still exist in the allowed list

is_ip_in_file("allow_list.txt", remove_list)
```

The detecting function confirmed presence of IDs to be removed, then the removing function gave me some details about their IPs and removed them .
Afterwards, the detecting function confirmed their removal.

```
True
unauthorized IP address found in the allowed list: 192.168.97.225
unauthorized IP address found in the allowed list: 192.168.158.170
unauthorized IP address found in the allowed list: 192.168.201.40
unauthorized IP address found in the allowed list: 192.168.58.57
False
```

For better clarity i decided to add a counter to count the number of occurrence and return the number before and after the removing function :

I added a counter that incremented in the loop each time the detection was true and didn't augment when it was false.

```
def is_ip_in_file(allowed_ip_file, array):
    counter = 0
    with open(allowed_ip_file, 'r') as file:
        file_content = file.read()
    list_of_ips = file_content.split()
    for ip in array:
        if ip in list_of_ips:
            counter += 1
        else:
            counter = counter
    print("number of unauthorized IP addresses found in the allowed list:
" + str(counter))
```

```
number of unauthorized IP addresses found in the allowed list: 4
unauthorized IP address found in the allowed list: 192.168.97.225
unauthorized IP address found in the allowed list: 192.168.158.170
unauthorized IP address found in the allowed list: 192.168.201.40
unauthorized IP address found in the allowed list: 192.168.58.57
number of unauthorized IP addresses found in the allowed list: 0
```